



Claude Code (БЕЗ ПЛАТНОГО ДОСТУПА)

Полное пошаговое руководство по настройке Claude Code, решению реальных проблем, созданию проектов и использованию Skills, MCP, плагинов, агентов и рабочих процессов без потери контроля.



МНЕНИЕ ИСКУССТВЕННОГО ИНТЕЛЛЕКТА
20 ИЮНЯ 2026 ГОДА



Проблема была не в написании кода

У меня на одном экране был открыт редактор, на другом — терминал, а в браузере — чат с искусственным интеллектом.

Приложение перестало собираться после небольшого изменения.

Я скопировал сообщение об ошибке из терминала, вставил его в чат, подожд ответа, скопировал предложенное исправление, вернулся в редактор, измени файл, снова запустил команду и получил другое сообщение об ошибке.

Затем ИИ попросил меня поделиться другим файлом.

Этот файл зависел от трех других файлов. Чат не знал структуру проекта. Он знал, что я изменил десять минут назад. Он давал разумные ответы, но всю работу между этими ответами приходилось выполнять мне.

Эту проблему решает Claude Code.

Он не сидит за пределами вашего проекта в ожидании скопированных фрагментов. Он открывается внутри проекта. Он сам считывает файлы. Он просматривает кодовую базу, проверяет историю Git, выполняет команды, находит реальную ошибку, редактирует нужный файл, снова запускает тест : сообщает, что изменилось.



108



5



14



Клод начинает работать там, где уже существует проблема.

Введите свой адрес электронной п

Подписка

На первом сеансе ощущения другие

Откройте папку проекта в терминале и запустите Claude Code.

Затем напишите:

```
Read this project. Do not change anything yet.
```

```
Explain:
```

- what the project does
- how the main folders connect
- where the app starts
- how data moves through it
- how tests are run
- anything that looks unfinished or risky

Клод начинает читать.

Он может изучить файл пакета, открыть основные файлы приложения, отследить импорт, найти обращения к базе данных, проверить тестовые папки и прочее, историю Git.

Здесь становится очевидной разница.

Вы не просите ИИ представить ваш проект по небольшому фрагменту кода. Он находится внутри реального проекта и осматривается вокруг.

Теперь вы можете спросить:

Или:

```
Find every place where customer prices are calculated.
```

Или:

```
Add CSV export to the reports page, but show me the plan before changing anything.
```

Claude Code — это все еще модель искусственного интеллекта. Она может неправильно понять требование, написать слабый код или пойти по ложному пути.

Ценность заключается в том, чтобы предоставить ей доступ к работе, а затем установить четкие ограничения на то, что она может делать.

В этом вам поможет данное руководство.

Введите свой адрес электронной п

Подписка

Что такое Код Клода на самом деле

Claude Code — это агентный рабочий инструмент Anthropic для программных проектов.

Он может:

- Чтение и поиск файлов
- Создание и редактирование кода
- Выполнение команд терминала
- Установка пакетов
- Запуск тестов

- Использование Git
- Просмотр изменений
- Работа с интегрированными средами разработки
- Подключение к внешним инструментам
- Делегирование работы другим агентам
- Запуск повторяющихся или запланированных процессов

It works mainly in the terminal, but it can also be used through supported editor desktop app, and remote interfaces.

The terminal is not the main idea.

The main idea is the loop:

**Read the real project → make a plan → do the work → run it → inspect the result
fix what failed**

A browser chatbot can explain how to perform those steps.

Claude Code can perform many of them itself.

Get it running first

On macOS, Linux, or WSL, run:

```
curl -fsSL https://claude.ai/install.sh | bash
```

On Windows PowerShell:

```
irm https://claude.ai/install.ps1 | iex
```

Now move into your project folder:

```
cd /path/to/your-project
```

Start Claude Code:

```
claude
```

Follow the sign-in process.

Then run:

```
/init
```

Claude will inspect the project and help create a `CLAUDE.md` file.

You should also use Git before allowing large changes:

```
git init
git add .
git commit -m "Save project before Claude Code changes"
```

Now you have a safe starting point.

Anthropic keeps the current installation instructions in the [official Claude Code guide](#).

The file Claude reads before you speak

It is a normal markdown file that tells Claude how this project should be handled.

Think of it as the note you would give a new developer before they touched the code.

A useful file might look like this:

Project

A wholesale bakery ordering system used by staff and business customers.

Stack

- Next.js
- TypeScript
- PostgreSQL
- Tailwind CSS
- Vitest

Important commands

- Start: npm run dev
- Test: npm test
- Lint: npm run lint
- Build: npm run build

Rules

- Do not change unrelated files.
- Ask before installing a new package.
- Never put passwords or API keys in source files.
- Follow the existing folder structure.
- Run tests before saying a task is complete.
- Explain any database change before applying it.

Product notes

- Staff and customer accounts have different permissions.
- Prices may differ by customer.
- Existing orders must never be changed during a product update.

Keep this file short.

Do not turn it into a storage room for every fact about the company. Claude read often, so it should contain only the information that matters across many tasks.

Task instructions belong elsewhere.

For a clearer explanation of the files around Claude Code, read [Learn Markdown CLAUDE.md, AGENTS.md, SKILL.md, DESIGN.md and More](#).



Learn Markdown Files: CLAUDE.md, AGENTS.md, SKILL.md, DESIGN.md and more

OPINION AI • 14 MIN.

[Read full story →](#)

Do not start by asking it to build everything

A common first prompt looks like this:

```
Build a complete SaaS app for restaurant owners.
```

Claude may happily start.

That is the danger.

It has to invent the product, users, database, permissions, design, payment flow, technical choices at the same time. You may receive a large amount of code before noticing that the basic direction is wrong.

Let it study the project

Explore the project and explain how this feature should fit into the current structure.
Do not edit anything.

Make it ask questions

Before planning, ask me about any requirement that is unclear or could change the design.

Ask for the plan

Show the exact files you expect to change, the steps you will take, the tests you will add, and the main risks. Do not write code yet.

Build one contained part

Implement only the password-reset request endpoint.

Do not build the email page yet.

Do not change unrelated authentication code.

Add focused tests and run them.

Check the result

Run the focused tests, full test suite, lint, and build.

Report the exact result of each command.

Do not say the task is complete if any check fails.

A wrong plan takes one message to repair.

A wrong implementation may take an hour to untangle.

[Subscribe](#)

Vibe coding is useful until the diff becomes unreadable

Vibe coding means describing the result in normal language and letting the agent build it.

There is nothing wrong with that.

It is one of the fastest ways to make a prototype, test an idea, or build a small internal tool.

The trouble begins when Claude edits thirty files and you accept the result because reviewing it feels harder than starting over.

Use Claude Code for speed, but keep the work visible.

Before finishing, ask:

Show me the complete diff.

Explain:

- why each file changed
- what assumptions you made
- what could still fail
- what I should manually test

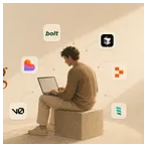
Then use the built-in review tools:

```
/diff  
/code-review  
/security-review
```

AI-written code is still code you own.

You need to understand what is entering the project.

Check the full guide on Vibe Coding:



Vibe Coding: The Complete Beginner's Guide (2026)

OPINION AI • 20 АПР.

[Read full story →](#)

The few commands that solve daily problems

You do not need to memorize every command.

These are enough for normal work:

`/init` creates or improves project instructions.

`/plan` lets Claude inspect and plan without making changes.

`/context` shows what is filling the context window.

`/compact` shortens a long session while keeping the important decisions.

`/clear` starts a clean conversation for a different task.

`/diff` shows the files and lines Claude changed.

`/rewind` returns to an earlier point when the session goes wrong.

`/model` changes the model used for the task

`/effort` changes how much reasoning Claude uses.

`/permissions` controls which actions Claude may perform.

`/doctor` checks whether Claude Code is installed and configured correctly.

`/usage` helps you understand usage and cost.

Do not keep one session alive for every task in the project.

When the work changes completely, use `/clear` or start a new session. Old discussion, logs, and failed attempts can make the next task harder.

Skills remove the instructions you repeat every week

Suppose every article you publish follows the same process:

1. Check the claims.
2. Remove robotic language.
3. Use short paragraphs.
4. Add useful links.
5. Prepare the final Substack format.

You can explain that process every time.

Or you can make it a Skill.

A Skill normally lives in:

```
.claude/skills/skill-name/SKILL.md
```

Examples

name: article-review

description: Review and prepare an article for publication on Opinion AI.

Article Review Process

1. Check every changing fact against a strong source.
2. Remove vague claims and inflated language.
3. Use short paragraphs and plain English.
4. Flag any section that sounds copied or machine-written.
5. Check every heading for usefulness.
6. Return a clean final draft and a short list of major changes.

Now you can run:

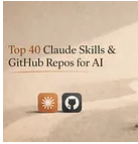
/article-review

Skills work well for:

- Code review
- Bug fixing
- Article editing
- Database migrations
- Product research
- Client reports
- Release checks
- Security review
- Deployment preparation

Claude Code performs the method on the current work.

For more examples, see [Top 40 Claude Skills and GitHub Repos That Actually Matter](#)



Top 40 Claude Skills & GitHub Repos for AI That Actually Matter R Now

OPINION AI • 12 APR.

[Read full story](#) →

MCP connects Claude to the rest of your work

Claude Code can already read local files and use the terminal.

MCP is for systems outside the local project.

MCP stands for Model Context Protocol. It gives Claude a standard way to use external tools and data.

You may connect it to:

- GitHub
 - Linear
 - Slack
 - Notion
 - Figma
 - Sentry
 - PostgreSQL
 - Supabase
 - Web browsers
-

Without MCP, you may copy a GitHub issue into Claude Code.

With the right MCP server, Claude can read the issue itself, inspect the related code, make the change, run tests, and prepare the pull request.

Check connected servers with:

```
claude mcp list
```

Or inside Claude Code:

```
/mcp
```

Start carefully.

A database connection should usually begin as read-only. A GitHub connection should not need access to every repository. A production server should not be the first environment you connect.

MCP gives Claude more reach. It also creates more ways for a bad instruction, untrusted plugin, or simple mistake to cause damage.

For a practical build guide, read [A Guide to Build MCP Servers, Plus 18 Awesome MCP Servers Worth Using](#).



A Guide to Build MCP Servers, plus 18 awesome MCP servers worth using

OPINION AI • 18 MIN.

[Read full story →](#)

Plugins can install a full working setup

- Skills
- Commands
- Subagents
- Hooks
- MCP settings
- Language tools

That makes plugins useful for teams.

A company could create one plugin containing its coding rules, review process, security checks, deployment commands, and project tools. A new developer could install that setup instead of building everything manually.

Open the plugin area with:

```
/plugin
```

Do not install plugins only because they look impressive.

A plugin may be able to run commands, read files, connect to services, or change Claude's behaviour. Check the source, permissions, maintainer, and included components.

I explained the full process in [How to Build Claude Plugins and Turn It Into a Small Business](#).



How to Build Claude Plugins and Turn it into Small Business

OPINION AI • 27 MAR

[Read full story](#) →

Hooks handle the rules Claude must never forget

Writing this in CLAUDE .md is useful:

Always run the formatter after editing a TypeScript file.

But it is still an instruction to the model.

A hook can run the formatter automatically after every edit.

Hooks can also:

- Block dangerous shell commands
- Stop writes outside approved folders
- Run tests after changes
- Check for secrets before a commit
- Save session logs
- Send a notification when work finishes

Use instructions for judgement.

Use hooks for fixed rules.

This is one of the main differences between using Claude as a helpful tool and building a working system around it. I covered that wider shift in [Turn Claude from Tool to System: 14 Lessons](#).



Turn Claude from Tool to System: 14 Lessons

OPINION AI • 26 MAP.

[Read full story](#) →

Subagents keep the mess out of the main sessio

Imagine asking Claude to inspect 200 files, read logs from five services, compare libraries, and search the Git history.

If all of that happens in the main session, the context fills with file contents, dead ends, and small findings.

A subagent can do that work separately.

Example:

```
Use a subagent to map the complete payment flow.
```

```
It should inspect the code, tests, database calls, and recent Git history.
```

```
Do not change anything.
```

```
Return only:
```

- the main flow
- key files
- likely failure points
- questions that still need answers

The subagent performs the messy investigation and returns a short report.

Your main session stays focused on the actual decision.

Subagents are useful for research, exploration, testing, security checks, and code review.

Agent teams and workflows are for genuinely large jobs

For example:

- One agent studies the frontend.
- One studies the backend.
- One checks security.
- One writes tests.
- One reviews the combined change.

A dynamic workflow goes further. Claude can create a script that divides a large sends many pieces to separate agents, and combines the results.

This can help with:

- Reviewing a large repository
- Checking hundreds of files
- Comparing many products or papers
- Running broad migrations
- Processing repeated items
- Researching several parts of a question at once

It can also consume a great deal of context and usage.

Do not begin with:

```
Run a workflow across my whole computer.
```

Begin with:

```
Create a workflow to review these ten files only.
```

For each file, report:

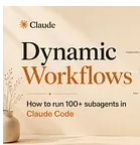
- bugs
- security risks
- missing tests

Use one final agent to merge duplicates and rank the five most serious findings.

Show me the workflow before running it.

Test the process on a small scope first.

My deeper guide is [Claude Dynamic Workflows: How to Run Hundreds of Parallel Sub-Agents](#).



Claude Dynamic Workflows: How to Run Hundreds of Parallel Sub-Agents

OPINION AI • 1 MIN.

[Read full story](#) →

Routines are where the work continues without you

A normal Claude Code session runs while you are there.

A Routine can run on a schedule, after an API call, or when something happens in GitHub.

Examples:

Every weekday morning, review new pull requests and prepare a short risk report.

When a new bug issue is opened, reproduce it, find the likely cause, and prepare a fix.

fix.

Every Friday, check outdated dependencies and open one report with the safest update order.

This is useful, but it needs tighter permissions than an attended session.

You are not present to stop a bad decision.

A Routine should have:

- A narrow job
- Clear success rules
- Limited repository access
- Limited credentials
- Safe stopping points
- A report of every action
- Human approval before high-risk changes

Automation becomes dangerous when the instruction is broad and the access is

Subscribe

Three prompts that produce better work

Build a feature

Read the relevant project files first.

Ask questions about unclear requirements.

- the implementation plan
- files you will change
- tests you will add
- security or data risks

Do not write code until I approve the plan.

After approval, keep the change tightly scoped.

Run tests, lint, and build.

Show the final diff.

Fix a bug

Reproduce the bug before changing code.

Read the real error, relevant source files, tests, and recent Git history.

Explain the root cause in plain language.

Write a failing test that proves the bug exists.

Make the smallest safe fix.

Run the focused test and full suite.

Do not change unrelated files.

Start a new project

I want to build [PRODUCT] for [USER].

The main problem is:

[PROBLEM]

The first version should only do:

2. [FEATURE]

3. [FEATURE]

Ask enough questions to remove major assumptions.

Then create:

- a short product brief
- user flow
- technical plan
- data model
- project structure
- security notes
- build order

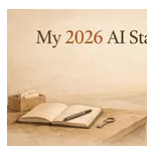
Do not write code yet.

The stack that works is usually the boring one

Claude Code can now sit inside a much larger system.

Memory files hold project rules. Skills hold repeated methods. MCP connects our tools. Plugins package setups. Hooks enforce fixed rules. Subagents isolate noisy Teams coordinate specialists. Workflows process large jobs. Routines run later.

I mapped those layers in [The AI Agent Stack You Actually Need in 2026](#).



The AI Agent Stack You Actually Need in 2026.

OPINION AI • 19 APR.

[Read full story](#) →

But the strongest setup is rarely the largest one.

It is usually this:

A clean Git repository.

One clear task.

A plan before risky changes.

Small diffs.

Tests that actually run.

A limited set of trusted tools.

A human who reads the result.

Claude Code is fast enough to create a large amount of work before you notice the direction is wrong.

The answer is not to slow the model down.

The answer is to give it a smaller target, better context, hard limits, and a clear target done.

Once that is in place, the terminal stops feeling like the difficult part.

The difficult part becomes deciding what deserves to be built.

And that is exactly where the human should remain.

Subscribe



108 Likes · 14 Restacks

Discussion about this post

Comments

Restacks

Write a comment...



DefenseHub 20 июн. *Edited*

♥ Liked by Opinion AI

I have been using various AIs for a while now and I am already starting to notice some different behavior between them. With the 3 I use the most for Coding: Gemini Pro 3.1, GPT 5.5, and C Code Opus 4.8. I have come to the following conclusion:

Claude - Excellent for designing complex structures, but sometimes careless with details.

GPT - Extremely careful with its code - I use it a lot to tweak website design and to review bugs, but limited for design.

Gemini 3.1 Pro - Construction worker - Excellent for day-to-day tasks and for following instructions from GPT and Claude.

I also highlight Gemini Flash, which is a hurricane for basic things.

This is my feeling from daily use. Regarding Claude, I tested Fable a little bit for code, I used it more to help me think. But just when it was getting good, they took it down.

That is it.

Congratulations on the excellent article.

♥ LIKE (5) 💬 REPLY

4 replies by Opinion AI and others

4 more comments...